

Name:	ID:	Section:
-------	-----	----------

Question 1 [C01] [10 Points]

Write a Java program to calculate electricity bills using a slab system. The program should take **total electricity consumption** (kWh), **connected load** (kW), and **customer type** (**1 = Residential**, **2 = Commercial**, **3 = Agricultural**) as input. Then calculate the bill as described below:

Energy Charge Slabs	Apply discount/rebate based on customer type
0 - 100 KWh → 7 Tk per KWh 101 - 300 KWh → 9 Tk per KWh - Above 300 KWh → 12 Tk per KWh	- Residential → 5% - Commercial → 3% - Agricultural → 10%
Demand charge = 30 Tk per kW of connected load.	

Bill Calculation:

- Energy charge = Calculated according to slab rates
- Demand charge = Connected load × 30
- Discount = (Energy charge + Demand charge) × Discount Rate
- Final bill = (Energy charge + Demand charge) – Discount

Input	output	Explanation
Total electricity consumption = 309 Connected load = 2 Customer type = 1	BILL DETAILS: Energy consumption charge = 2608 Tk Demand charge = 60 Tk Discount = 133.4 Tk Total bill = 2534.6 Tk	Energy consumption charge: 100*7 = 700tk (for first 100kwh) 200*9 = 1800 (for next 200 kwh) 9*12= 108 (above 300kwh) Total energy consumption charge = 700+1800+108 = 2608 Demand charge = 2*30 = 60 Discount / rebate = (2608+60)*5% = 133.4 Total bill = ((2608+ 60)-(2608+ 60)*.05) = 2534.6

Name :	ID:	Section:
--------	-----	----------

Question 1 [C01] [10 Points]

Write a Java program to calculate water bills using a slab system. The program should take **total water consumption** (in cubic meters, m³), **connection size**, and **customer type** (**1 = Residential**, **2 = Commercial**, **3 = Agricultural**) as input. Then calculate the bill as described below:

Water Charge Slabs	Apply discount/rebate based on customer type
0 - 50 m³ → 10 Tk per m³ 51- 100 m³ → 15 Tk per m³ - Above 100 m³ → 20 Tk per m³	- Residential → 5% - Commercial → 3% - Agricultural → 10%
Connection charge = 50 Tk per unit	

Bill Calculation:

- Water charge = Calculated according to slab rates
- Connection charge = Connection size × 50
- Discount = (Water charge + Connection charge) × Discount Rate
- Final bill = (Water charge + Connection charge) – Discount

Input	output	Explanation
Total water consumption = 109 Connected size = 2 Customer type = 1	BILL DETAILS: Water consumption charge = 1430 Tk Connection charge = 100 Tk Discount = 76.5 Total bill = 1453.5 Tk	Water consumption charge: 50*10 = 500tk (for first 50m ³) 50*15 = 750 (for next 50 m ³) 9*20= 180 (above 100 m ³) Total water consumption charge = 500+750+180 = 1430 Connection charge = 2*50=100 Discount / rebate = (1430+100)*5% = 76.5 Total bill = ((1430+100)-(1430+100)*.05) = 1453.5

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

SET - [A]

SOLUTION

```
import java.util.Scanner;
public class ElectricityBill {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input
        System.out.print("Total electricity consumption = ");
        int consumption = sc.nextInt();

        System.out.print("Connected load = ");
        int load = sc.nextInt();

        System.out.print("Customer type = ");
        int type = sc.nextInt();

        double energyCharge = 0;

        // Slab Calculation
        if (consumption <= 100) {
            energyCharge = consumption * 7;
        } else if (consumption <= 300) {
            energyCharge = (100 * 7) + (consumption - 100) * 9;
        } else {
            energyCharge = (100 * 7) + (200 * 9) + (consumption - 300) * 12;
        }

        // Demand Charge
        double demandCharge = load * 30;

        double total = energyCharge + demandCharge;

        // Discount
        double discount = 0;
        if (type == 1) {
            discount = total * 0.05;    // Residential
        } else if (type == 2) {
            discount = total * 0.03;    // Commercial
        } else if (type == 3) {
            discount = total * 0.10;    // Agricultural
        } else {
            discount = 0;
        }

        double finalBill = total - discount;

        // Output
        System.out.println("\n----- BILL DETAILS -----");
        System.out.println("Energy Charge = " + energyCharge + " Tk");
        System.out.println("Demand Charge = " + demandCharge + " Tk");
    }
}
```

```

        System.out.println("Discount = " + discount + " Tk");
        System.out.println("Total Bill = " + finalBill + " Tk");
    }
}

```

RUBRIC

SL	Points to Meet	Marks [10]
1	Proper Java Syntax (Importing Scanner, creating class and main method and creating Scanner Object)	2
2	Taking 3 inputs properly	1
3	Calculate energy consumption charge properly	2
4	Calculate demand charge properly	1
5	Calculate discount properly	2
6	Calculate final bill properly	1
7	Proper print statements according to sample input and output	1

SET - [B]

SOLUTION

```

import java.util.Scanner;
public class WaterBill {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input
        System.out.print("Total water consumption = ");
        int consumption = sc.nextInt();

        System.out.print("Enter connection size = ");
        int size = sc.nextInt();

        System.out.print("Customer type = ");
        int type = sc.nextInt();

        double waterCharge = 0;

        // Slab Calculation
        if (consumption <= 50) {
            waterCharge = consumption * 10;
        } else if (consumption <= 100) {
            waterCharge = (50 * 10) + (consumption - 50) * 15;
        } else {
            waterCharge = (50 * 10) + (50 * 15) + (consumption - 100) * 20;
        }

        // Connection Charge
        double connectionCharge = size * 50;

        double total = waterCharge + connectionCharge;

        // Discount
        double discount = 0;

        if (type == 1) {
            discount = total * 0.05;    // Residential
        } else if (type == 2) {
            discount = total * 0.03;    // Commercial
        } else if (type == 3) {
            discount = total * 0.10;    // Agricultural
        } else {
            discount = 0;
        }

        double finalBill = total - discount;

        // Output
        System.out.println("\n----- BILL DETAILS -----");
        System.out.println("Water Consumption charge = " + waterCharge + " Tk");
        System.out.println("Connection Charge = " + connectionCharge + " Tk");
        System.out.println("Discount = " + discount + " Tk");
        System.out.println("Total Bill = " + finalBill + " Tk");
    }
}

```

RUBRIC		
SL	Points to Meet	Marks [10]
1	Proper Java Syntax (Importing Scanner, creating class and main method and creating Scanner Object)	2
2	Taking 3 inputs properly	1
3	Calculate Water consumption charge properly	2
4	Calculate Connection charge properly	1
5	Calculate discount properly	2
6	Calculate final bill properly	1
7	Proper print statements according to sample input and output	1



CSE110: Programming Language I (Lab)
Spring 2026
Lab Exam - 02
Duration: 15 Minutes

A

Name:	ID:	Section:
-------	-----	----------

Question 1 [C01] [10 Points]

You are provided with a number N from the user. Your task is to print a pattern of numbers in descending order, where each row starts from n and decreases by 1 until the row number. The total number of rows will be equal to the value of N.

Sample Input	Sample Output
4	4 43 432 4321
6	6 65 654 6543 65432 654321



Name:	ID:	Section:
-------	-----	----------

Question 1 [C01] [10 Points]

You are given a number N from the user. Your task is to print a pattern where the number of characters in each row increases by 1, and the symbols alternate between two characters. The pattern should continue for N rows.

Sample Input	Sample Output
4	& \$\$ &&& \$\$\$\$
6	& \$\$ &&& \$\$\$\$ &&&&& \$\$\$\$\$\$

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

SET - [A]

SOLUTION

```
import java.util.Scanner;

public class SetA {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int N = scanner.nextInt();
        for (int i = 1; i <= N; i++) {
            for (int j = N; j >= N - i + 1; j--) {
                System.out.print(j);
            }
            System.out.println();
        }
    }
}
```

RUBRIC

SL	Points to Meet	Marks [10]
1	Take input from user	1
2	Properly handle outer loop	2
3	Properly handle inner loop	4
4	Correct output pattern generation	3
5		

SET - [B]

SOLUTION

```
import java.util.Scanner;

public class SetB {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int n = scanner.nextInt();

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= i; j++) {
                if (i % 2 == 1) {
                    System.out.print("&");
                } else {
                    System.out.print("$");
                }
            }
            System.out.println();
        }
    }
}
```

RUBRIC

SL	Points to Meet	Marks [10]
1	Take input from user	1
2	Properly handle outer loop	2
3	Properly handle inner loop	4
4	Correct output pattern generation	3
5		

Name:	ID:	Section:
-------	-----	----------

Question 1 [C02] [10 Points]

You are building a text filtering tool for a data cleaning application. Users often input strings where words are separated by a special character. Some characters in each word appear only once (considered noise), while others appear multiple times (considered significant).

Your task is to develop a program that removes all unique (single-occurrence) characters from each word while keeping all occurrences of characters that appear more than once.

The program should:

- Read a string containing multiple words separated by a user-specified character
- Read the separator character
- Split the string manually (no built-in split)
- For each word, remove characters that appear only once
- Print the filtered words

Input	Output
heellllloo-worrrld -	The words are: eellllloo rr



Name:	ID:	Section:
-------	-----	----------

Question 1 [C02] [10 Points]

You are developing a text analyzer where users input strings with repeated characters and special separators. In this case, characters that appear only once are considered meaningful, while repeated characters are treated as noise.

Your task is to develop a program that keeps only the characters that appear exactly once in each word and removes all repeated characters.

The program should:

- Read a string containing multiple words separated by a user-specified character
- Read the separator character
- Split the string manually (no built-in split)
- For each word, keep only characters that appear once
- Print the cleaned words

Input	Output
heellllloo#worrlld #	The words are: h wold

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

SET - [A]

SOLUTION

```
import java.util.Scanner;

public class Question1 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String text = sc.nextLine();
        char separator = sc.nextLine().charAt(0);

        // add separator at end
        text = text + separator;

        System.out.println("The words are:");

        String word = "";

        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);

            if (ch == separator) {
                String result = "";

                for (int j = 0; j < word.length(); j++) {
                    char current = word.charAt(j);
                    int count = 0;

                    for (int k = 0; k < word.length(); k++) {
                        if (word.charAt(k) == current) {
                            Count++;
                        }
                    }

                    // keep only repeated characters
                    if (count > 1) {
                        result = result + current;
                    }

                    System.out.println(result);
                    word = "";
                } else {
                    word = word + ch;
                }
            }
        }
    }
}
```

RUBRIC		
SL	Points to Meet	Marks [10]
1	Correct input handling (string + separator)	2
2	Proper manual splitting using loop (no built-in)	2
3	Correct logic for counting character frequency	2
4	Correct filtering condition (Set A: >1, Set B: ==1)	2
5	Correct output formatting and handling all words	2

SET - [B]**SOLUTION**

```
import java.util.Scanner;

public class Question2 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String text = sc.nextLine();
        char separator = sc.nextLine().charAt(0);

        // add separator at end
        text = text + separator;

        System.out.println("The words are:");

        String word = "";

        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);

            if (ch == separator) {
                String result = "";

                for (int j = 0; j < word.length(); j++) {
                    char current = word.charAt(j);
                    int count = 0;

                    for (int k = 0; k < word.length(); k++) {
                        if (word.charAt(k) == current) {
                            count++;
                        }
                    }

                    // keep only unique characters
                    if (count == 1) {
                        result = result + current;
                    }
                }

                System.out.println(result);
                word = "";
            } else {
                word = word + ch;
            }
        }
    }
}
```

RUBRIC		
SL	Points to Meet	Marks [10]
1	Correct input handling (string + separator)	2
2	Proper manual splitting using loop (no built-in)	2
3	Correct logic for counting character frequency	2
4	Correct filtering condition (Set A: >1, Set B: ==1)	2
5	Correct output formatting and handling all words	2

Name:	ID:	Section:
-------	-----	----------

Question 1 [C03] [10 Points]

You are given an integer array and you need to take a number as input from the user. Your task is to update the array by replacing the duplicate values of the array with zero. The updated array should be sorted in ascending order. Finally, you have to print the position of the number (you took as input from the user) from the updated array. Print **"Not found"** if the number is not in the array.

Can't use built-in sort for the problem. Can't create a new array for this problem.

Input	Output
<pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=2</pre>	<pre>Updated array: 6, 2, 0, 1, 0, 9, 0 Sorted array: 0,0,0,1,2,6,9 Output: Position of 2 = 5</pre>
<pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=5</pre>	<pre>Updated array: 6, 2, 0, 1, 0, 9, 0 Sorted array: 0,0,0,1,2,6,9 Output: Position of 5 = Not found</pre>

Name:	ID:	Section:
-------	-----	----------

Question 1 [C03] [10 Points]

You are given an integer array and you need to take a number as input from the user. Your task is to create a new array that will contain only the unique (distinct) elements of the given array. The new array should be sorted in descending order. Finally, you have to print the position of the number (you took as input from the user) in the new array. Print **"Not found"** if the number is not in the array.

Can't use built-in sort for the problem.

Input	Output
<pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=6</pre>	<pre>New array: 6, 2, 1, 9 Sorted array: 9,6,2,1 Output: Position of 6 = 2</pre>
<pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=7</pre>	<pre>New array: 6, 2, 1, 9 Sorted array: 9,6,2,1 Output: Position of 7 = Not found</pre>

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

SET - [A]

SOLUTION

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        int[] arr = {6, 2, 6, 1, 2, 9, 1};
        int elem = 5;

        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < i; j++) {
                if (arr[i] == arr[j]) {
                    arr[i] = 0;
                    break;
                }
            }
        }

        System.out.println("Updated Array:");
        for (int num : arr) {
            System.out.print(num + " ");
        }

        for (int i = 0; i < arr.length - 1; i++) {
            int minIndex = i;

            for (int j = i + 1; j < arr.length; j++) {
                if (arr[j] < arr[minIndex]) {
                    minIndex = j;
                }
            }

            int temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }

        int pos = -1;
        System.out.println("\nSorted Array:");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
            if (arr[i] == elem){
                pos = i+1;
            }
        }
        if(pos == -1){
            System.out.printf("\nOutput: \nPosition of %s = Not found", elem);
        }else{
            System.out.printf("\nOutput: \nPosition of %s = %s", elem,pos);
        }
    }
}
```

```
}
```

RUBRIC

SL	Points to Meet	Marks [10]
1	Handle Duplicates by replacing with 0 in the array	3
2	Sort the array in ascending order	3
3	Search and find the position	2
4	Follow the constraint (No built-in sort + No new array)	1
5	Maintain Output format (updated array, sorted array, output position)	1

SET - [B]**SOLUTION**

```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        int[] arr = {6, 2, 6, 1, 2, 9, 1};
        int elem = 6;

        int[] newArr = new int[arr.length];
        int count = 0;

        for (int i = 0; i < arr.length; i++) {
            boolean isDuplicate = false;

            for (int j = 0; j < count; j++) {
                if (arr[i] == newArr[j]) {
                    isDuplicate = true;
                    break;
                }
            }

            if (!isDuplicate) {
                newArr[count] = arr[i];
                count++;
            }
        }

        System.out.println("\nNew array:");
        for (int i = 0; i < count; i++) {
            System.out.print(newArr[i] + " ");
        }

        for (int i = 0; i < newArr.length - 1; i++) {
            int maxIndex = i;

            for (int j = i + 1; j < newArr.length; j++) {
                if (newArr[j] > newArr[maxIndex]) {
                    maxIndex = j;
                }
            }

            int temp = newArr[i];
            newArr[i] = newArr[maxIndex];
            newArr[maxIndex] = temp;
        }

        int pos = -1;
        System.out.println("\nSorted Array:");
        for (int i = 0; i < count; i++) {
            System.out.print(newArr[i] + " ");
            if (newArr[i] == elem){
                pos = i+1;
            }
        }
        if(pos == -1){
            System.out.printf("\nOutput: \nPosition of %s = Not found", elem);
        }
    }
}

```

```

    }else{
        System.out.printf("\nOutput: \nPosition of %s = %s", elem,pos);
    }
}
}

```

RUBRIC

SL	Points to Meet	Marks [10]
1	Create unique elements in new array	3
2	Sort the array in descending order	3
3	Search and find the position	2
4	Follow the constraint (No built-in sort)	1
5	Maintain Output Format (new array, sorted array, output position)	1